FACULTAD DE INGENIERIA Y ARQUITECTURA

MATERIA

GESTION DE SERVIDORES WEB

DOCENTE:

ING. ALBERTO TURCIOS ORTEZ

INTEGRANTES EQUIPO 6:

EVELIN CAROLINA MORALES FLORES – U20250919

MARLON STIVEN RAMOS VÁSQUEZ – U20210287

KARLA ALEXANDRA CASTRO RIVERA – U20241427

ACTIVIDAD:

LABORATORIO 2

ENTREGA: 12 DE OCTUBRE DE 2026

Laboratorio 2: Configuración de Servidores Web

Proyecto de Infraestructura Web "Dulce Hogar" — Equipo 6

FASE 1: DIMENSIONAMIENTO COLABORATIVO

1. SUFICIENCIA DE RECURSOS Y FACTOR PICO

Los 2 vCPU y 4 GB de RAM son suficientes para el tráfico diario normal de 500 visitas, ya que representa una carga promedio muy baja para Apache. Para el pico de 3,000 visitas también son suficientes, siempre que Apache esté bien optimizado, ya que equivale a pocos usuarios concurrentes por segundo.

- **Cálculo del Factor Pico:**
    - **Factor Pico = (Tráfico Pico en Días Festivos) / (Tráfico Diario Normal)**
    - **Factor Pico = 3,000 visitas / 500 visitas = 6x**

El factor pico es de 6x, lo que indica que en días festivos la demanda se sextuplica respecto a un día común.

2. Estrategia de Escalado: Vertical vs. Horizontal

Recomendamos el ESCALADO VERTICAL. Para una PyME como "Dulce Hogar", incrementar la CPU o RAM del mismo servidor en días festivos es la opción más económica y rápida de implementar. El escalado horizontal aumentaría innecesariamente los costos en balanceadores de carga y requeriría un tiempo de configuración mucho mayor para el volumen de tráfico estimado.

3. Monitoreo y Recurso Crítico

Vigilaríamos PRIMERO la memoria RAM. Si la RAM se agota por múltiples conexiones de Apache, el servidor Ubuntu recurre a la memoria SWAP en disco, volviendo el sistema extremadamente lento. En segundo lugar monitorearíamos la CPU para evitar saturación de peticiones. La red y el disco SSD de 40 GB son de menor riesgo inmediato para esta carga.

4. DECISIÓN EN EQUIPO: TABLA COMPARATIVA (NUBE VS. SERVIDOR LOCAL)

Decisión: Para el desarrollo y evaluación del laboratorio se utiliza Servidor Local (WSL2) por costo $0 y control de pruebas. Sin embargo, para producción comercial de "Dulce Hogar", se recomienda un servidor en la Nube (VPS) para garantizar disponibilidad 24/7.

Criterio	**Servidor Local (WSL2)**	**Servidor en la Nube (VPS)**
Costo Inicial	**$0 (usa el hardware existente)**	**Muy bajo (pago por uso mensual)**
Escalabilidad	**Limitada al hardware físico de la PC**	**Alta y rápida (ajuste inmediato)**
Mantenimiento	**Alto (requiere energía/red local)**	**Bajo (gestionado por el proveedor)**
Control	**Total y físico sobre la máquina**	**Lógico total mediante SSH/Root**

FASE 2: CONFIGURACIÓN BÁSICA DEL SERVIDOR WEB (APACHE)

1. ¿Qué muestra la página por defecto de Apache? Capturen pantalla.

El servidor muestra la pantalla de bienvenida "Apache2 Ubuntu Default Page". Esto ocurre porque es la configuración de fábrica de Apache apuntando a la ruta '/var/www/html/'. Esto demuestra que el servicio está activo, sin errores, y listo para que en la Fase 3 se configuren los VirtualHosts que enlazarán la ruta correcta de los archivos web del proyecto.



2. ¿Qué diferencia hay entre sites-available y sites-enabled?

- sites-available: Guarda todos los archivos de configuración (.conf) creados, estén encendidos o apagados.
- sites-enabled: Guarda solo accesos directos de los sitios que están "activos" (los que encendemos con a2ensite). Apache solo lee esta carpeta para funcionar.

3. Ejecuten sudo apache2ctl configtest. ¿Qué reporta? ¿Por qué es importante antes de reiniciar?

Reporta Syntax OK si no hay errores en el código. Es vital ejecutarlo antes de reiniciar porque si hay un error tipográfico en un archivo .conf y reiniciamos a ciegas, el servidor se caerá y todas las páginas dejarán de funcionar.

4. Reto: si Apache no arrancara, ¿qué 3 pasos seguirían para diagnosticar el problema?

1. Ejecutar sudo systemctl status apache2 para ver si el servicio marca error y por qué.
2. Ejecutar sudo apache2ctl configtest para verificar que no hay errores de código en los VirtualHosts.
3. Revisar los registros del sistema con sudo tail /var/log/apache2/error.log para ver el motivo exacto de la caída (ej. un puerto ya ocupado).

FASE 3 - VIRTUALHOST

1. ¿Qué hace ProxyPass /api http://localhost:9000/api?

El comando ProxyPass permite que Apache actúe como un intermediario (Reverse Proxy), reenvíe las solicitudes HTTP que recibe en la ruta /api hacia nuestra API de Node.js, la cual está funcionando de forma interna en el puerto 9000. De esta manera, los clientes pueden consumir los servicios de la API a través del servidor web principal sin necesidad de conectarse directamente al puerto del backend.

2. ¿Qué diferencia hay entre ProxyPass y ProxyPassReverse?

- ProxyPass: Se encarga del flujo de ida; redirige y envía las peticiones del cliente que entran por Apache hacia el servidor backend (Node.js).
- ProxyPassReverse: Se encarga del flujo de vuelta; modifica y ajusta las cabeceras HTTP de respuesta que emite el backend para asegurar que el cliente siga viendo las rutas a través de Apache y no descubra la estructura o puertos internos.

En resumen: ProxyPass envía las solicitudes al backend y ProxyPassReverse reescribe las respuestas del backend para mantener la coherencia de la navegación a través del proxy.

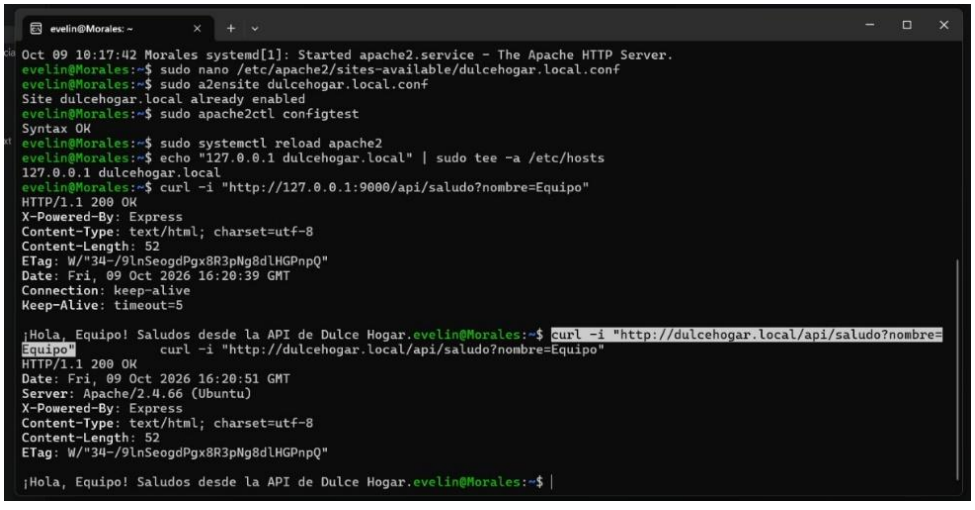
3. Reto: ¿Qué pasaría si configtest falla? ¿Cómo lo depurarían entre todos?

Si el comando sudo apache2ctl configtest falla, significa que Apache detectó un error de sintaxis o directivas mal escritas en sus archivos de configuración, por lo que no se debe recargar ni reiniciar el servicio para evitar caídas del servidor web.

Proceso de depuración:

- Leer detenidamente el mensaje de error que arroja la consola para identificar exactamente en qué archivo y número de línea se encuentra la falla.
- Revisar el archivo de configuración afectado (por ejemplo, dulcehogar.local.conf), verificando la correcta apertura y cierre de etiquetas, la sintaxis de las rutas y la ortografía de las directivas.
- Comprobar que las dependencias lógicas y los módulos requeridos (como proxy, proxy_http y headers) se encuentren correctamente habilitados.
- Aplicar la corrección correspondiente en el archivo de texto.
- Volver a ejecutar sudo apache2ctl configtest tantas veces como sea necesario hasta que el sistema devuelva el mensaje *Syntax OK*.
- Recargar Apache de forma segura utilizando sudo systemctl reload apache2 y verificar la conectividad mediante curl.
- ¿Por qué el comando curl devuelve correctamente el saludo de la API a través de Apache?

Porque Apache recibe la solicitud HTTP inicial en el puerto 80, reconoce la regla del Reverse Proxy configurada para la ruta /api y la reenvía de manera transparente hacia la API de Node.js en el puerto 9000. La API procesa el parámetro recibido (el nombre), genera la respuesta con el saludo y se la devuelve a Apache, el cual la entrega de vuelta al cliente de forma exitosa.



FASE 4 - HTTPS

Pregunta 1: ¿Por qué certbot no funciona para el dominio dulcehogar.local?

Certbot utiliza entidades certificadoras públicas (como Let's Encrypt) que requieren validar la propiedad del dominio a través de internet. Debido a que "dulcehogar.local" es un dominio de uso exclusivamente interno (local) y no cuenta con registros DNS públicos, los servidores externos no pueden conectarse a él para realizar la verificación exigida.

Pregunta 2: ¿Qué diferencia hay entre usar un certificado autofirmado vs uno firmado por mkcert?

Un certificado autofirmado no tiene el respaldo de ninguna entidad certificadora, por lo que el navegador lo bloquea inmediatamente con una advertencia roja de seguridad. Por el contrario, la herramienta mkcert genera su propia Autoridad Certificadora (CA) local, la cual se instala manualmente en el almacén de "Entidades de certificación raíz de confianza" de Windows. Esto obliga al navegador a confiar en el certificado como si fuera emitido por una entidad comercial, cerrando el candado de seguridad.

Pregunta 3: Reto: Si el equipo decidiera desplegar la aplicación en un dominio público real (por ejemplo dulcehogar.com.sv), ¿qué pasos adicionales o diferentes deberían realizar?

Se descartaría por completo el uso de mkcert. Los pasos correctos serían:

1. **Configurar los registros DNS (Tipo A) del dominio real para que apunten a la IP pública de nuestro servidor.**
2. **Permitir el tráfico abriendo los puertos 80 y 443 en el firewall de red hacia el servidor.**
3. **Instalar y ejecutar la herramienta certbot para generar, validar a través de internet e instalar automáticamente un certificado SSL real y válido a nivel global.**

Nota técnica sobre la evidencia adjunta:

En la captura de pantalla entregada en la ruta /api/saludo?nombre=Equipo, se visualiza un error HTTP 503 (Servicio no disponible). Este es el comportamiento esperado y confirma que la directiva ProxyPass del Virtual Host está operando correctamente: Apache recibe la conexión segura HTTPS en el puerto 8900 y deriva el tráfico hacia el puerto 9000. Al no encontrarse en ejecución el servicio backend (API de Node.js de la Fase 3) en el entorno local, el proxy retorna correctamente el estado 503, validando la configuración**.**

```
<IfModule mod_ssl.c>
<VirtualHost *:8900>
    ServerName dulcehogar.local
    DocumentRoot /var/www/dulcehogar.local

    SSLEngine on
    SSLCertificateFile /etc/apache2/ssl/dulcehogar.local/dulcehogar.local.pem
    SSLCertificateKeyFile /etc/apache2/ssl/dulcehogar.local/dulcehogar.local-key.pem

    ProxyPreserveHost On
    ProxyPass /api http://localhost:9000/api
    ProxyPassReverse /api http://localhost:9000/api
</VirtualHost>
</IfModule>
```


Servicio 503 No disponible
https://dulcehogar.local:8900/api/saludo?nombre=Equipo
Marlon Ramos
PDF
SERVIDORES
TIENDA
EMAILS
Servicio no disponible
El servidor temporalmente no puede dar servicio a tu solicitud debido a tiempos de inactividad o capacidad de mantenimiento problemas. Por favor, inténtalo de nuevo más tarde.
Apache/2.4.66 (Ubuntu) Server at dulcehogar.local Port 8900

CONCLUSIÓN DEL EQUIPO

La realización de este laboratorio nos permitió comprender la importancia de una correcta arquitectura de servidores web. A través de las cuatro fases, logramos evolucionar desde un entorno HTTP básico y vulnerable, hasta una infraestructura segura utilizando un proxy inverso (ProxyPass) y certificados de seguridad SSL locales mediante mkcert. Comprobamos que separar los servicios (Apache en los puertos 80/8900 y Node.js en el puerto 9000 interno) no solo optimiza el rendimiento, sino que protege la estructura interna de la aplicación, garantizando que el usuario final navegue por un entorno encriptado (HTTPS) y seguro.